RGS.Center

Подробная техническая документация проекта

Структура • реализованные возможности • незавершённые части • направления улучшения

Параметр	Значение
Состояние исходников	03 августа 2026
Базовый коммит	d5f25ad - replace RoboPLC monitoring with LenovoPC
Технология	Java 26, Spring Boot 4.1, Thymeleaf, jSerialComm
Объём	90 основных Java-файлов; 22 тестовых; 8 651 строк Java
Проверка	59 тестов; BUILD SUCCESS
Назначение	Сервис управления роботизированным гамма-спектрометром

Документ описывает фактически присутствующий код. Предполагаемые функции явно отделены от реализованных; эмулятор измерительного сценария не считается управлением реальным оборудованием.

1. Резюме состояния проекта

RGS.Center уже имеет рабочий каркас многопоточного Spring-приложения: устройства получают отдельные сервисы, соединения запускаются вместе с приложением, команды и ответы проходят через ограниченные очереди, а последние состояния публикуются в общей витрине. Реализованы несколько реальных протокольных адаптеров и web-страницы мониторинга.

Что уже является сильной основой

- Единый Connection/Reader/Writer-слой для LAN, TCP и COM.
- Отдельные RX/TX-потоки на физическое подключение; безопасная остановка через ExecutorService.
- Ограниченные очереди, защищённый кольцевой буфер и сменные декодеры кадров.
- Автоматический запуск ManagedDeviceConnection через DeviceConnectionManager.
- Витрина текущих состояний и централизованный сервис типизированных настроек.
- Полноценные протокольные реализации GDRM, SHP, Gamma-PRO и базовая Dobot-телеметрия.
- Эмулятор полного измерительного цикла и 59 автоматических тестов.

Главный вывод

Проект находится на стадии интеграционного прототипа. Коммуникационная инфраструктура в основном готова, но производственный автоматический сценарий, постоянное хранение, безопасность, весы, полный протокол сканера и пользовательские Lua-команды манипулятора ещё не завершены.

Критический путь до опытной эксплуатации

Приоритет	Результат	Зависимость
P0	Зафиксировать Lua/TCP-команды PLC и привязать 9 шагов цикла к реальным сервисам	Протокол Валтера
P0	Довести сканер: свет/trigger/stop/retry/валидация	Проверка RS-232 на стенде
P0	Реализовать весы	Модель, COM-параметры, протокол
P1	Добавить БД: настройки, измерения, история, аудит	Выбор PostgreSQL/другой БД
P1	Аутентификация и роли Operator/Engineer/Admin	Политика доступа
P1	Интеграционные испытания на полном стенде	Доступность оборудования

2. Назначение, пользователи и системный контекст

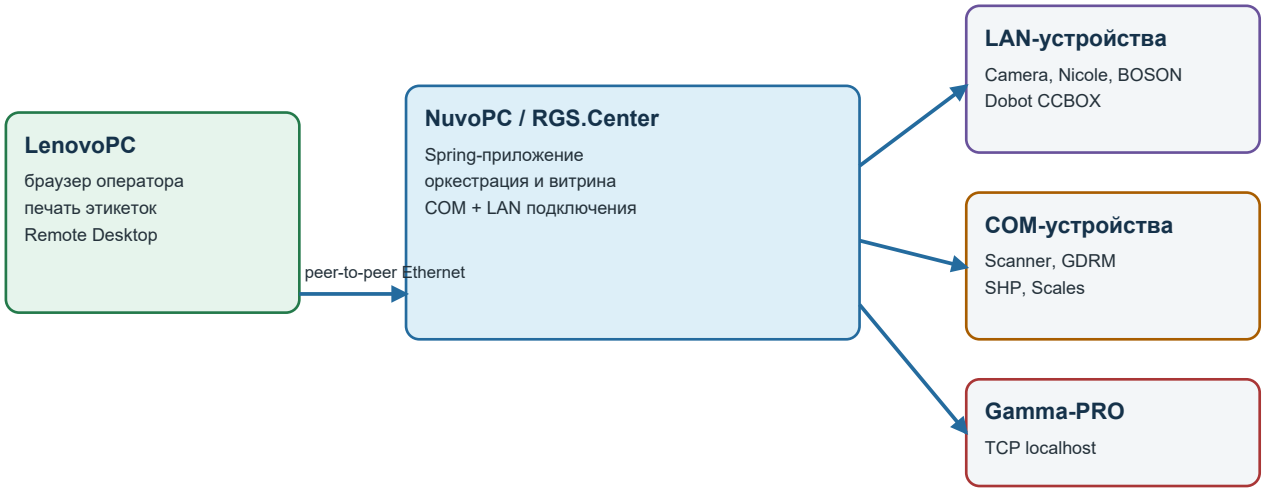
Назначение

RGS.Center должен работать сервисом на NuvoPC внутри роботизированного гамма-спектрометра. Приложение опрашивает оборудование, хранит оперативные статусы, управляет продолжительным измерительным процессом и предоставляет web-интерфейс.

Роли пользователей

Роль	Основные задачи	Статус реализации
Оператор	Загрузка до 50 объектов, формирование списка, START, контроль процесса, просмотр истории	UI мониторинга и эмулятор есть; реальный сценарий и история отсутствуют
Сервисный инженер	Подробные статусы, ручные команды, диагностика портов, журналы	Снимки worker и логи есть; отдельный сервисный UI отсутствует
Администратор	Учётные записи, роли, параметры системы	Не реализовано

Контекст оборудования



LenovoPC является рабочим местом конечного пользователя и сетевым выходом комплекса. NuvoPC выполняет RGS.Center и непосредственно связан с LAN/COM-оборудованием. Dobot Studio используется на LenovoPC для программирования PLC/CCBOX, но автоматическое управление должно идти из RGS.Center по TCP-протоколу.

3. Технологический стек и модель развёртывания

Область	Используемая технология	Комментарий
Язык	Java 26	Указано в pom.xml; перед production следует утвердить поддерживаемую JDK/LTS
Framework	Spring Boot 4.1.0	Web MVC, DI, lifecycle, scheduling
Web	Thymeleaf + HTML/JS	Фрагменты страниц обновляются без REST API
COM	jSerialComm 2.11.4	GDRM, SHP, barcode; Scales зарезервированы
TCP/LAN	java.net.Socket / InetAddress reachability	Gamma-PRO, Dobot, сетевые ping-сервисы
Параллелизм	ExecutorService, blocking queues, locks, atomics	Отдельные потоки и безопасная остановка
Тесты	JUnit 6 / Spring test	59 тестов, включая протоколы и очереди
Сборка	Maven Wrapper	Spring Boot executable jar

Ожидаемое размещение

- NuvoPC: RGS.Center, Gamma-PRO, локальные COM-подключения, LAN к приборам.
- LenovoPC: браузер оператора, печать этикеток, моделирование, Remote Desktop, Dobot Studio на этапе разработки.
- Физические IP/COM-параметры задаются в application.properties и должны быть вынесены во внешний production-профиль.

Текущие сетевые значения по умолчанию

Узел	Адрес/порт	Назначение
Camera	192.168.1.13	ping; получение изображения ещё не реализовано
BOSON	192.168.1.14	ping
Nicole	192.168.1.15	ping
LenovoPC	192.168.201.20	peer-to-peer ping
Dobot CCBOX	192.168.5.1:29999 / :30005	Dashboard-команды / feedback 200 ms
Gamma-PRO	127.0.0.1:37569	Scenarium TCP

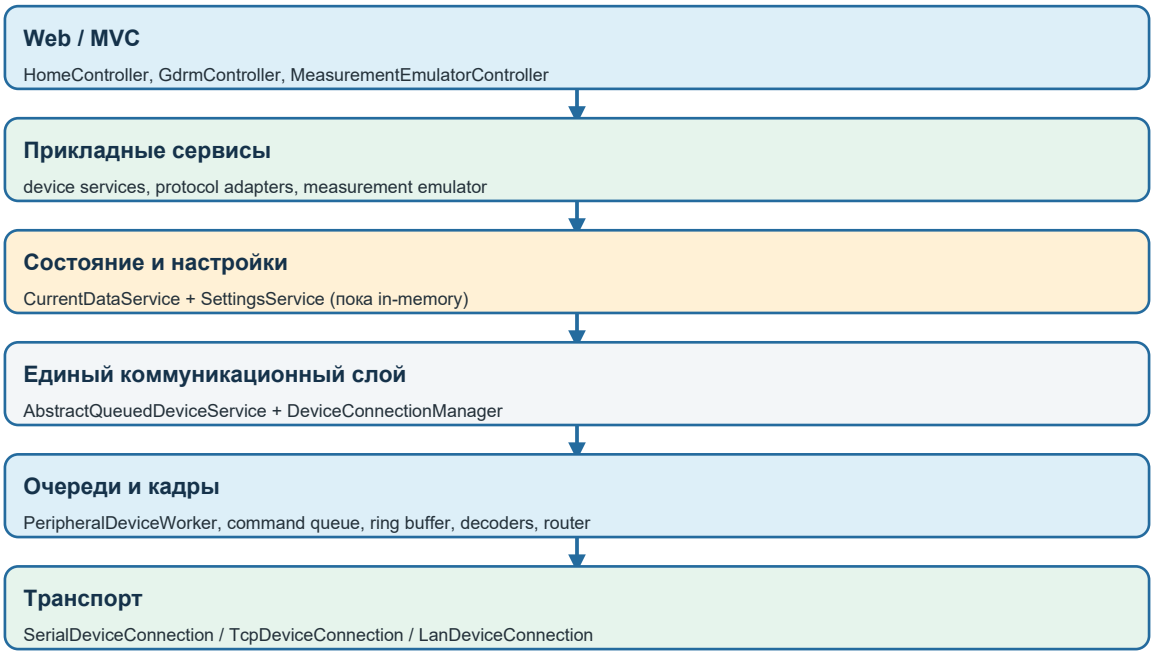
4. Структура исходного проекта

RGSCenter/
center/
pom.xml
src/main/java/com/rgs/center/
configuration/ планировщик фоновых задач
connection/ абстракции COM/LAN/TCP и lifecycle
controller/ главная страница и /devices
peripheral/ очереди, буфер, worker, декодеры, router
service/ настройки, витрина, ping, scanner, network devices
gdrm/ протокол, сервис и web-представление GDRM
shp/ бинарный протокол и опрос SHP
gammapro/ TCP-протокол и управление набором спектра
manipulator/ Dobot Dashboard + поток feedback
measurement/ эмулятор автоматического сценария
objects/ общие DTO, коды и статусы
src/main/resources/
application.properties
templates/ Thymeleaf-страницы и обновляемые фрагменты
src/test/java/ 22 тестовых класса
Docs/ руководства оборудования
output/pdf/ итоговые отчёты

Пакеты и ответственность

Пакет	Классов	Ответственность / ключевые элементы
connection	12	AbstractDeviceConnection; COM/LAN/TCP; Reader/Writer; DeviceConnectionManager
peripheral	18	worker, очереди, ByteRingBuffer, 4 декодера, IncomingFrameRouter, snapshots
service	18	SettingsService, CurrentDataService, ping-сервисы, barcode, network devices
gdrm	9	COM-протокол, измерение, operational data, /gdrm
gammapro	7	Scenarium TCP, heartbeat, START/STOP/WAIT_MEAS_END
manipulator	7	Dashboard protocol, 1440-byte feedback, двухканальный сервис
measurement.emulator	7	состояния, проверки, таблица объектов, 9-шаговый цикл
objects	5	Device, DeviceCode, SettingRecord, SettingType
shp	4	C0/0C framing, датчики, snapshot
controller/configuration	2	web-маршруты и scheduler pool

5. Внутренняя архитектура



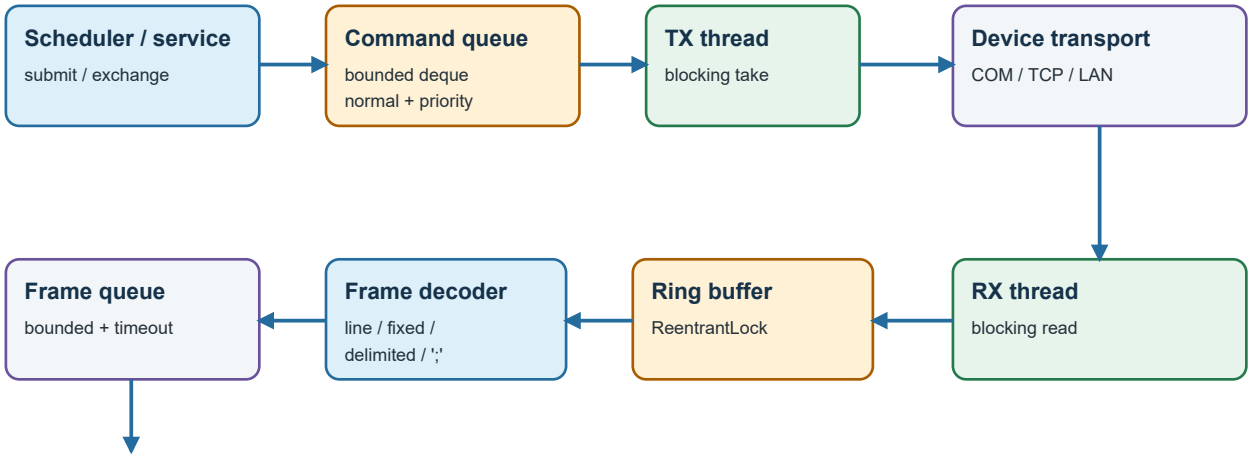
Основные архитектурные правила

- Контроллеры не читают физические порты: они получают готовые снимки сервисов.
- Один физический канал принадлежит одному PeripheralDeviceWorker.
- Команды копируются и ставятся в ограниченную TX-очередь; ответы выделяются декодером в RX-потоке.
- Сбой одного устройства не блокирует запуск web-приложения и остальных соединений.
- Настройки адресуются строковыми кодами; типы и значения проверяются SettingsService.
- CurrentDataService хранит только последнее оперативное состояние, не историю.

Жизненный цикл

Spring создаёт все сервисы. DeviceConnectionManager получает список ManagedDeviceConnection и запускает их через SmartLifecycle. При завершении приложения соединения закрываются в обратном порядке. Периодические задачи выполняются ThreadPoolTaskScheduler с пулом из 8 потоков.

6. Обслуживание очередей, потоков и буферов



Потребитель получает готовый кадр; ожидание блокирующее, без активного Thread.sleep.

Что реализовано вместо опасных конструкций GDRMonitor

Проблемная конструкция	Реализация RGS.Center	Эффект
Несинхронизированный LinkedList	LinkedBlockingDeque / LinkedBlockingQueue	Потокобезопасность и backpressure
Циклы Thread.sleep	blocking take/poll + Duration timeout + Condition	Нет активного ожидания
Thread.stop()	shutdownNow + close transport + awaitTermination	Предсказуемое освобождение ресурсов
Незащищённый ring buffer	ByteRingBuffer + ReentrantLock	Целостность RX-данных
Ожидание освобождения порта	Единственный владелец соединения + очередь команд	Нет конкуренции за COM/TCP
Неограниченный рост	Ограниченные очереди; drop oldest с метрикой	Контролируемая деградация

Поддерживаемые способы выделения кадров

- LineFrameDecoder - строки CR/LF.
- FixedLengthFrameDecoder - фиксированный бинарный размер, например Dobot 1440 байт.
- DelimitedFrameDecoder - стартовый и конечный байты, например SHP C0/0C.
- TerminatedFrameDecoder - кадр до одного терминатора, например Dobot ';'.
- IncomingFrameRouter - независимые маршруты COMMAND_RESPONSE, TELEMETRY, ALERT, UNKNOWN.

7. Настройки и витрина текущих данных

SettingsService

Сервис реализует две потокобезопасные карты: код -> тип и код -> значение. Начальные значения загружаются из Spring properties. Методы load/dump используют SettingRecord в форме будущей таблицы БД: STRING_ID, TYPE, BOOLEAN_VALUE, LONG_VALUE, DOUBLE_VALUE, STRING_VALUE. Реального repository/JDBC/JPA слоя пока нет.

Группа	Примеры настроек
LAN ping	camera, boson, nicole, lenovo-pc, ping.timeout.millis
Dobot	host, command/feedback port, timeouts, stale interval, DI bits
Barcode	COM7, baud 9600, read timeout, maximum code length
GDRM	COM3, initial baud 9600, response/poll timeouts
SHP	COM5, baud 9600, response/poll timeouts
Gamma-PRO	127.0.0.1:37569, windows-1251, acquisition timeouts
Scales	только зарезервированное имя COM-порта

CurrentDataService

Для каждого DeviceCode создаётся ConcurrentHashMap параметров. Общие поля: pinged, interaction_lastdate_in/out, data_available, last_data, last_error. Протокольные сервисы добавляют собственные поля - координаты, DI/DO, режим робота, измерения SHP и т.д. Снимки выдаются как неизменяемые копии.

Ограничение: витрина не гарантирует атомарный согласованный снимок нескольких полей одного устройства и не сохраняет историю. Для production нужен типизированный агрегат устройства или версия снимка плюс транзакционная запись истории.

8. Матрица интеграции оборудования

Устройство	Канал	Сейчас реализовано	Статус / следующий шаг
Camera	LAN 192.168.1.13	Отдельный ping-сервис, /devices	Частично: нужен getImage и список нескольких камер
LenovoPC	LAN 192.168.201.20	Отдельный queued ping каждые 10 с	Мониторинг готов; адрес подтвердить на стенде
Nicole	LAN 192.168.1.15	Отдельный ping-сервис	Нужен getData и разбор статуса OK
BOSON	LAN 192.168.1.14	Отдельный ping-сервис	Нужен getData и BOSON.Status=READY
Dobot manipulator	TCP 29999 + 30005	Dashboard-команды; 1440-byte feedback; pose, DI/DO, mode, command id; reconnect	Нужен пользовательский Lua-протокол и команды 9 шагов
Barcode scanner	RS-232 COM7	Постоянный worker, line framing, readBarcode, витрина	Нет PWRLED/PWRAIM/SYN T/U, retry и бизнес-валидации
GDRM	COM3	Команды, измерение каналов, operating data, snapshot, HTML endpoint	Проверить на реальном приборе и добавить историю
SHP	COM5	Бинарные кадры, 6 датчиков, resistance, reconnect, snapshot	Проверить калибровку/единицы на стенде
Gamma-PRO	TCP 127.0.0.1:37569	Heartbeat, tracts, SET_TRACT, SET_PRESET, START/STOP, WAIT_MEAS_END	Связать с production orchestrator и хранением спектров
Scales	COM (не задан)	DeviceCode и setting placeholder	Не реализовано: нужен протокол и сервис

RoboPLC удалён из актуальной модели. Управление контроллером манипулятора сосредоточено в RoboticManipulatorStreamService; отдельный RGS-plc следует возвращать только после уточнения его самостоятельной роли и протокола.

9. Подробности реализованных протокольных сервисов

9.1 RoboticManipulatorStreamService

- Два независимых PeripheralDeviceWorker: Dashboard 29999 и feedback 30005.
- DashboardProtocol проверяет ASCII-команду и разбирает ErrorID,{values},echo;.
- FeedbackParser валидирует длину 1440 и TestValue, читает little-endian поля.
- В витрину публикуются x/y/z/rx/ry/rz, joint_1..6, DI/DO, robot_mode, enabled, running, error_status, safety_state, current_command_id.
- Завершение движения ожидается Condition-сигналом при совпадении ResultID и RobotMode=5.
- Watchdog отмечает stale feedback и перезапускает failed-каналы.

9.2 Gamma-PRO

Сервис сериализует протокольные операции ReentrantLock. В idle выполняется GET_TRACTS_NAME. Для измерения передаются SET_TRACT и SET_PRESET, затем START и ожидание WAIT_MEAS_END. Состояния: IDLE, RUNNING, COMPLETED, FAILED. STOP доступен отдельно.

9.3 GDRM

COM-сервис собирает дозиметрические измерения по каналам и operating data, публикует связь, адрес, серийный номер, firmware, напряжение, ток, температуру и прочие параметры. /gdrm показывает человекочитаемую таблицу вместо JSON.

9.4 SHP

Сервис формирует команды чтения шести параметров и резистора обратной связи, использует DelimitedFrameDecoder, защищает poll от перекрытия и после трёх последовательных ошибок закрывает соединение для переподключения.

9.5 Barcode scanner

BarcodeScannerGetDataAdapter читает один кадр из RX-очереди, обновляет витрину и передаёт ошибку вызывающему коду. Однако текущая версия не выполняет требуемую последовательность выключения при старте, включения подсветки/trigger, stop и повторной попытки.

10. Web-интерфейс и endpoint’ы

Метод	Путь	Назначение	Формат
GET	/	Главная страница и меню	HTML
GET	/devices	Список Camera, LenovoPC, Nicole, BOSON, Dobot, Scanner, SHP, Gamma-PRO	HTML
GET	/devices/status	Фрагмент таблицы; браузер обновляет раз в 10 с	Thymeleaf fragment
GET	/gdrm	Страница измерений GDRM	HTML
GET	/gdrm/status	Обновляемый фрагмент GDRM	Thymeleaf fragment
GET	/measurement-emulator	Панель эмулятора	HTML
GET	/measurement-emulator/status	Текущий снимок эмулятора	Thymeleaf fragment
POST	/measurement-emulator/reset	Сброс	fragment
POST	/measurement-emulator/detect-objects	Имитация автоопределения	fragment
POST	/measurement-emulator/start	Запуск цикла	fragment
POST	/measurement-emulator/checks/{code}/toggle	Имитация отказа/восстановления	fragment
POST	/measurement-emulator/leakage/toggle	Имитация тока утечки	fragment

Механизм обновления

Страницы используют JavaScript setInterval(loadStatus, 10000) и загружают серверный HTML-фрагмент. Meta refresh и отдельный REST API не используются. Это просто и соответствует текущему объёму, но для real-time событий манипулятора и процесса лучше добавить SSE или WebSocket.

Пробелы интерфейса

- Нет аутентификации и разграничения ролей.
- Нет страницы детальной диагностики очередей/соединений всех устройств.
- Нет ручного сервисного управления с подтверждениями опасных команд.
- Нет истории измерений, поиска, экспорта и печати этикеток.
- Нет production-дизайна ошибок, уведомлений и подтверждения оператором.

11. Алгоритм измерения и эмулятор

Эмулятор моделирует проверки связи, начальные состояния, контрольные параметры, выставление высокого напряжения, ток утечки, изображение камеры, формирование списка объектов и последовательную обработку. Он не отправляет команды физическим устройствам.



Состояния эмулятора

Состояние	Смысл
PRECHECK	Проверки оборудования, начальных состояний и параметров
PARALLEL_PREPARATION	Высокое напряжение, ток утечки, камера
WAITING_FOR_OBJECTS	Ожидание списка объектов
READY_TO_START	Все условия выполнены; START доступен
RUNNING	Последовательное выполнение 9 шагов по каждому объекту
ALERT	Оборудование/параметр вышли из нормы
COMPLETED	Все объекты обработаны

Что требуется для превращения эмулятора в orchestrator

- Заменить tick-переходы на async-команды сервисов с timeout/cancel/retry.
- Хранить сценарий и объект измерения в БД с идемпотентными шагами.
- Добавить compensation/safe-stop: выключить scanner light, остановить Gamma-PRO, перевести robot в безопасное состояние.
- Возобновлять процесс после рестарта только из проверенной контрольной точки.
- Сохранять протокол каждого шага и привязку barcode/weight/spectrum к объекту.

12. Качество, тестирование и метрики

Метрика	Значение
Основные Java-файлы	90
Тестовые Java-файлы	22
Строки main Java	7 236
Строки test Java	1 415
Текстовые символы src/main + src/test	386 545
Автоматические тесты	59; failures 0; errors 0
Последняя проверка	mvnw test - BUILD SUCCESS

Покрытые области

- Кольцевой буфер, очередь команд, worker и frame router.
- Lifecycle менеджера подключений и LAN transport.
- Протоколы Dobot Dashboard/feedback, GDRM, SHP, Gamma-PRO.
- Витрина, настройки, ping и независимость сервисов.
- Barcode getData и автомат состояний эмулятора.
- Spring context smoke test.

Недостающие проверки

- Hardware-in-the-loop тесты на реальных COM/TCP устройствах.
- Контрактные тесты против эмуляторов каждого прибора.
- Нагрузочные тесты очередей и длительного feedback-потока Dobot.
- Тесты reconnect/backoff при разрыве кабеля и смене COM-порта.
- End-to-end web-тест полного операторского сценария.
- Проверки безопасности и прав доступа после появления Spring Security.

13. Что ещё необходимо реализовать

Область	Необходимый результат	Критичность
Orchestrator	Production state machine 9 шагов, timeout/retry/cancel/safe state	P0
Dobot Lua	Контракт команд, ответы, коды ошибок, completion events	P0
Scanner	PWRAIM0, PWRLED0/2, SYN T/U, retry, validation	P0
Scales	COM connection, protocol, stable weight, validation	P0
Safety	Interlocks, HV leakage, hatch DI, emergency stop policy	P0
Persistence	Settings table, measurement/object/step history, audit	P1
Security	Users, password policy, roles, sessions, CSRF, audit	P1
Camera	getImage, multiple camera registry, object detection integration	P1
Nicole/BOSON	Real getData adapters and semantic readiness fields	P1
Operations	Actuator/health/metrics, log rotation, alerts, backups	P1
UX	Operator workflow, service dashboard, history/export	P2
CI/CD	Repeatable JDK, pipeline, artifacts, release configuration	P2

Обязательные уточнения от команды оборудования

- Файл пользовательского протокола Lua TCP-сервера Dobot.
- Фактические IP всех интерфейсов и схема маршрутизации peer-to-peer.
- Назначение DI/DO: концевики люка, gripper, interlocks.
- Модель и протокол весов, COM-параметры, формат стабильного значения.
- Формат и правила валидации штрих-кода.
- Требования к хранению спектров и связи с результатом Gamma-PRO.

14. Что можно улучшить в существующем коде

Направление	Предлагаемое изменение	Польза
Типизация витрины	DeviceSnapshot вместо произвольных Map	Компиляционная проверка и атомарные снимки
Настройки	@ConfigurationProperties + validation; DB repository	Меньше строковых ошибок, внешние профили
Reconnect	Exponential backoff + jitter + circuit breaker	Нет частых повторов при длительном отказе
Startup	Параллельный bounded-start соединений	Недоступный прибор не задерживает запуск
Команды	Typed Command/Response и correlation id	Безопасная маршрутизация параллельных сообщений
Ошибки	Единая иерархия error code + operator message + technical cause	Понятные уведомления и аудит
Observability	Micrometer metrics, Spring HealthIndicator, queue depth gauges	Проактивная диагностика
Web updates	SSE для состояний и событий сценария	Меньше polling и быстрее реакция
Testing	Device simulators + integration profile	Повторяемость без стенда
Docs	Обновить PERIPHERAL_ARCHITECTURE.md и ADR	Снижение рассинхронизации документации

Наблюдаемые технические риски

- Java 26 и Spring Boot 4.1 могут затруднить развёртывание, если production-среда стандартизована на LTS JDK.
- Текущая Map-витрина допускает чтение полусогласованного набора параметров во время обновления.
- Физические соединения открываются при Spring startup; лог тестов содержит ожидаемые ошибки отсутствующего оборудования.
- ICMP/reachability зависит от ОС и firewall; для критичных устройств лучше прикладной heartbeat.
- Нет persistence: рестарт теряет статусы, текущий сценарий и результаты.

15. Рекомендуемый план развития

Этап	Содержание	Критерий готовности
1. Контракты	Утвердить Lua, scales, scanner validation, DI/DO, safety	Версионированные protocol specs и test vectors
2. Симуляторы	TCP/COM simulators для Dobot, scanner, scales, Gamma-PRO	CI выполняет полный цикл без оборудования
3. Устройства P0	Scanner control + Scales + Dobot typed commands	Каждый адаптер проходит контрактные тесты
4. Orchestrator	Durable 9-step workflow, cancel/retry/safe-stop	Один и несколько объектов проходят цикл
5. Persistence	Настройки, партии, объекты, шаги, результаты, audit	Рестарт продолжает/безопасно завершает процесс
6. Security/UI	Роли, операторский и сервисный интерфейс, уведомления	Доступ ограничен ролью; ошибки понятны
7. Стенд	HIL, fault injection, endurance, acceptance tests	Согласованный протокол приёмочных испытаний
8. Release	JDK/runtime, installer/service, log rotation, backup	Воспроизводимый подписанный release artifact

Предлагаемый Definition of Done для устройства

- Документированная версия протокола и настройки.
- Типизированные команды/ответы и проверка границ значений.
- Отдельный worker, bounded queues, timeout, reconnect, safe shutdown.
- Витрина состояния, last interaction, error code и metrics.
- Unit + simulator integration + hardware acceptance tests.
- Страница диагностики и эксплуатационная инструкция.

16. Как добавлять новое устройство

Шаги реализации

№	Действие	Контрольный результат
1	Добавить DeviceCode и SettingsService/application.properties	Устройство появляется в витрине
2	Выбрать SerialDeviceConnection, TcpDeviceConnection или LAN	Transport открывается и закрывается
3	Выбрать decoder и PeripheralWorkerConfiguration	Кадры выделяются без потерь/переполнения
4	Создать service от AbstractQueuedDeviceService	Сервис автоматически найден manager'ом
5	Реализовать protocol encoder/parser	Test vectors проходят
6	Добавить poll/getData/commands и обновление CurrentDataService	Есть snapshot и last error
7	Добавить UI/endpoint только для готовых снимков	Web-поток не читает порт
8	Добавить unit, simulator и HIL тесты	Сбой устройства изолирован

Правила проектирования

- Никогда не открывать один COM/TCP-порт из нескольких сервисов.
- Не использовать Thread.stop, busy-wait и неограниченные очереди.
- Не смешивать transport framing и бизнес-валидацию.
- Не выполнять длительный exchange из MVC controller.
- Всегда задавать timeout, возможность cancel и безопасное состояние после ошибки.
- Не считать ping достаточной проверкой semantic readiness устройства.

17. Эксплуатация и диагностика

Запуск и проверка

```
cd C:\Projects\RGSCenter\center
mvnw.cmd test
mvnw.cmd spring-boot:run

Web:
http://localhost:8080/devices
http://localhost:8080/gdrm
http://localhost:8080/measurement-emulator

Log:
center/logs/rgscenter.log
```

Диагностический снимок worker

PeripheralWorkerSnapshot содержит state, глубину command/frame queues, размер RX buffer, bytes sent/received, frames received/dropped, timestamps и lastError. Эти поля рекомендуется вывести на отдельную страницу сервисного инженера и в Micrometer.

Типовой порядок поиска неисправности

- Проверить фактический IP/COM и отсутствие владельца порта в другой программе.
- Проверить state и lastError worker, затем timestamps и bytes sent/received.
- Если bytesReceived растут, проверить decoder и protocol parser.
- Если framesDropped растут, увеличить скорость потребителя или пересмотреть queue capacity.
- Проверить firewall/маршрутизацию для peer-to-peer LAN и TCP-портов.
- После трёх повторяемых сбоев сохранить лог, snapshot и входной кадр для анализа.

18. Ключевые файлы и источники знаний

Файл	Назначение
connection/DeviceConnectionManager.java	Автоматический lifecycle всех подключений
peripheral/PeripheralDeviceWorker.java	RX/TX, queues, buffer, metrics, shutdown
peripheral/IncomingFrameRouter.java	Разделение ответов, telemetry, alerts
service/SettingsService.java	Типы и значения настроек
service/CurrentDataService.java	Оперативная витрина
manipulator/RoboticManipulatorStreamService.java	Dobot command + feedback
gammapro/GammaProService.java	Управление набором спектра
gdrm/GdrmService.java	Опрос GDRM
shp/ShpService.java	Опрос SHP
measurement/emulator/MeasurementEmulatorService.java	Модель процесса
resources/application.properties	Текущие адреса, COM и интервалы

Локальные документы оборудования

- Dobot TCP_IP Remote Control Interface Guide V4.6.0.
- Dobot Nova Series User Guide V1.5.
- Gamma-PRO manual, глава 17.
- SHP protocol fv2.4.
- Honeywell/Vuquest scanner manual, глава 11.
- GDRM технологическая инструкция и исходный проект GDRMonitor.

Заключение

RGS.Center уже отделил протоколы устройств от web-слоя и заменил небезопасные потоковые приёмы на управляемые очереди и lifecycle. Следующая ценность создаётся не расширением каркаса, а соединением готовых адаптеров в надёжный, сохраняемый и безопасный производственный сценарий.